

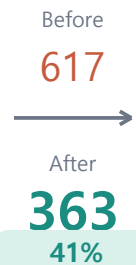
한 문장

리소스의 실제 수명은 5종 이상인데 표현 수단이 2개(Global/Scene)뿐이라 애매한 것이 전부 “안전한 쪽=Global”로 도피해 영구 상주하던 구조를, (key,type) refCount 레지스트리 + 수명 스코프로 재구축해 상주 메모리를 회수했다. — 수동 Release 코드는 한 줄도 추가하지 않았다.

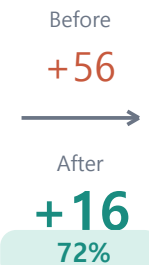
진입 Global 메모리



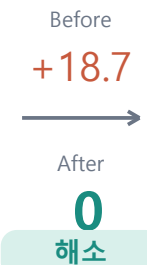
전체 합계



Global 단조 증가



던전 왕복 영구 증가



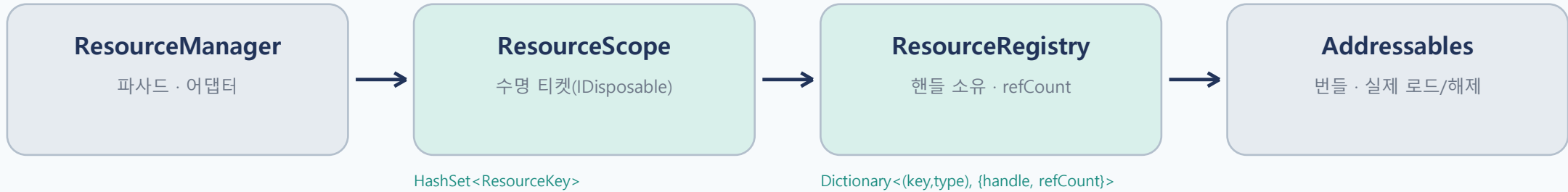
대가 — 정직하게

lazy화는 공짜가 아니다. 진입 308MB ↔ 팝업 첫 오픈 +14~20ms.

“상시 부담(진입 메모리)을 일회성 지연(첫 오픈)으로 바꾼 거래” — 재오픈은 회귀 없음(~9ms).

핵심 주장은 수치가 아니라 방법론과 구조. HP바 누수는 개인 실수가 아니라 2버킷 구조가 만든 선택이었다.

호출부 130곳 무수정 (어댑터)



1 소유는 레지스트리 한 곳

스코프는 티켓만 든다.
"누가 Release하나"
문제 자체가 소멸.

2 스코프 내 중복 = refCount 1

HashSet.Add 반환값으로 판단.
refCount = 소유한
서로 다른 스코프 수.

3 취소 ≠ 해제

토큰은 기다림 중단만.
수명은 스코프 소유 →
use-after-release 구조적 불가.

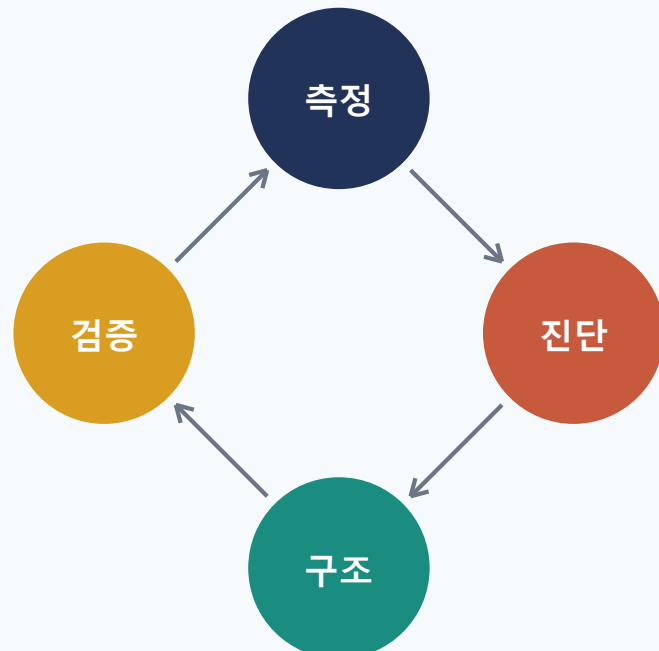
수명 계층



순서 규칙 (Party-Popup)

① 인스턴스 파괴 → ② 스코프 Dispose
역순이면 살아있는 인스턴스가 메시·텍스처를
참조하는 동안 핸들만 반납 → refCount는
풀려도 실제 언로드가 안 일어난다.

던전 스코프는 안 만들었다 — 목표가 이미 달성돼(근거 기반 축소)



숫자를 믿기 전에 도구를 검증했다

- 측정 도구가 아틀라스를 ×6.8 과대 계상하던 계통 오차를
에디트/플레이 모드 교차 검증으로 발견·보정(436MB → 실제 64MB).
- 계측 이관 후엔 isGlobal 한 줄만 토글해 단일 변수 A/B —
재측정 Before가 v2와 512.8MB 소수점까지 일치 = 도구 등가성 실측.
- “에디트 모드와 플레이 모드가 상반된 결과를 냈다” — 도구도 교차검증 대상.

계측이 도구였다 — 드러난 기존 버그 (뿌리: 에셋 수명 ≠ 인스턴스 수명)

● UI_LootNotification

DDOL 인스턴스 + Scene 스코프 에셋 = use-after-release. 왕복 후 재로드도 안 됨

● PreloadAsync 영구 정지

로드 실패 시 인스턴스 null인데 플래그만 내려가 WaitUntil 무한 대기

● _isLoadingPopup 미복구

조기 반환 시 플래그가 true로 남아 이후 모든 팝업 영구 차단

● 취소 정산 유령 핸들 (셀프리뷰)

스코프만 롤백/레지스트리 미롤백 → 어느 스코프도 소유 않는 refCount≥1 누수

“메모리가 줄었다”는 지표가 실은 누수를 못 센 결과일 수 있다 — R6에서 합계 보존(0.3MB)으로 유령이 회계에 드러남을 실증.

“Addressables가 이미 refCount인데 왜 위에 또?”

Addressables refCount는 핸들 단위 수동 짝맞추기 — 호출부가 Release를 '기억'해야 하고, 그 기억의 실패가 HP바 누수였다. 스코프는 기억을 구조(수명 단위 자동 정산)로 대체한 것.

“본인이 만든 문제를 본인이 고친 것 아닌가?”

맞다, 숨기지 않는다. 2버킷은 초기 설계였고 계측으로 한계를 발견 → 개인의 주의력이 아니라 구조로 재발 차단. 레거시 개선은 실무의 일상, 이 프레이밍이 성장 서사.

“1인 프로젝트에 오버엔지니어링 아닌가?”

팝업 세션 +287MB라는 측정치가 필요성을 증명. 반대로 던진 스코프·프레임 예산 스트리밍·DDOL 대공사는 측정 근거로 제외 = 필요한 것만 했다는 증거.

어필 포인트 (결과물보다 방법론)

- 측정→진단→구조→검증 루프, 각 결정에 근거
- 계측이 기존 버그 3건을 드러냄 (수명 불일치)
- 셀프 코드리뷰로 잔여 결함 2건 자가 발견·수정
- 수동 Release 0줄 — refCount 자동 정산
- split-brain 버그를 숨기지 않고 서술 (정직함)

한계 (반드시 동반)

메모리 절대값은 에디터 측정(CPU 사본 과대) —

개발 빌드 실측 0회 → 절대값 주장 금지, 증감·비율만.

단, Use Existing Build = 실제 번들 확인으로

증감·비율은 신뢰도 “강”.

히치 ms는 $n=1\sim3$, 15ms 이하 차이 주장 금지.